Real-Time Hand Sign Language Detection and Responding System Using YOLO-v11

Subject: Construction IT and Automation - 2025/12/15

Name: Nguyen Chi Tien

Master_student

CONTENT

I. INTRODUCTION

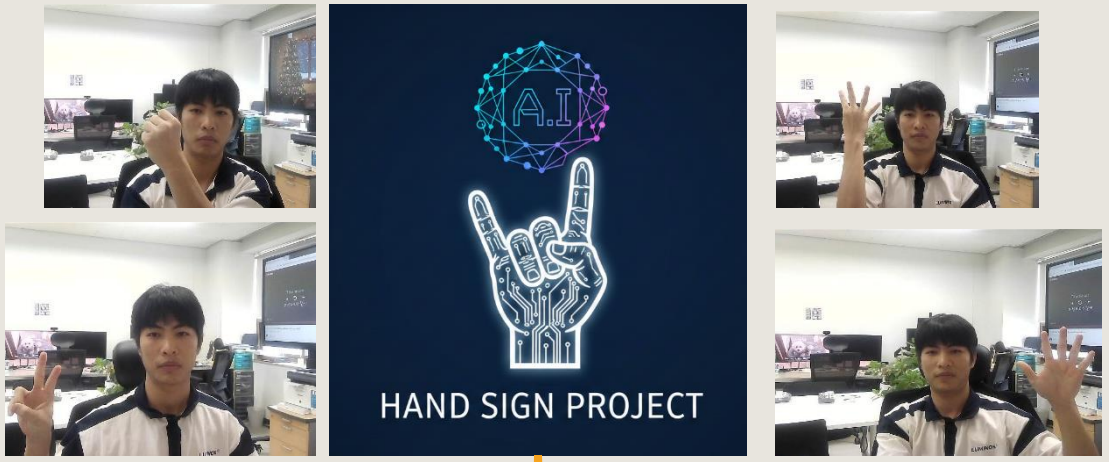
II. DATA ACQUISITION

III. EVALUATION

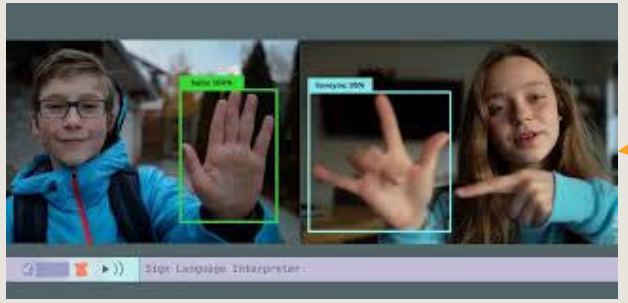
IV. EXPERIMENTAL RESULT

V. CONCLUSION

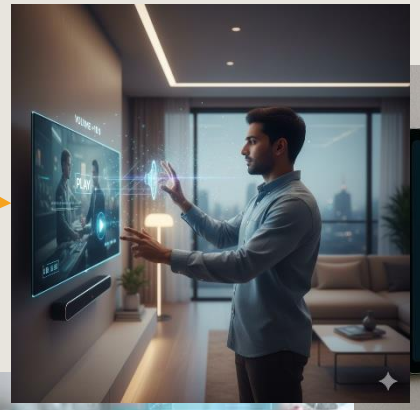
I. INTRODUCTION



Communication Asisstance

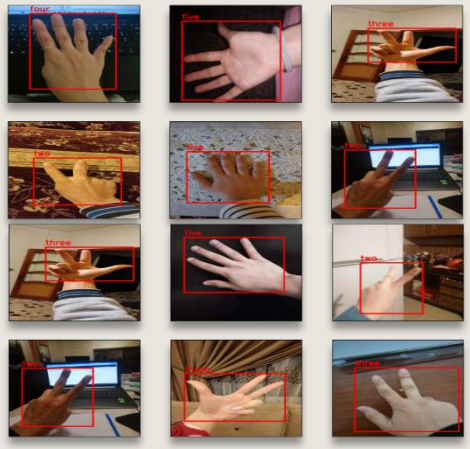


Human-Computer Interaction



A. Workflow

Data collection



Annotation

ultralytics

YOLO11

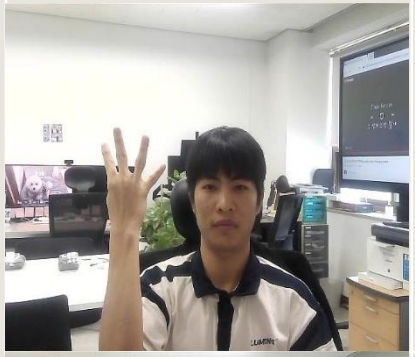
Training with Yolo for interaction

Webcam

Testing

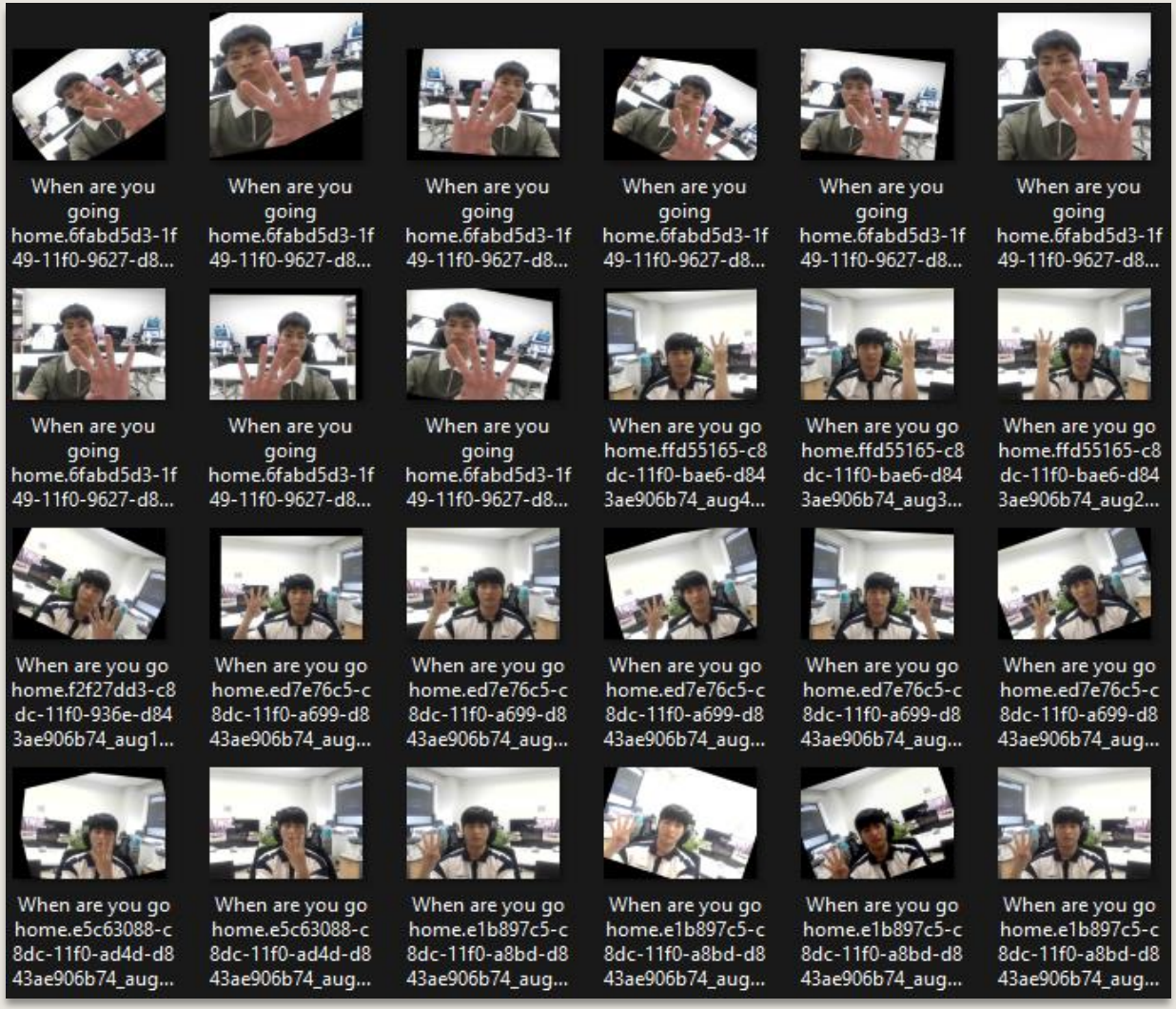
B. Data Collection

Image	Class
	Hello, I can not speak.
	Where are you from?
	How are you today?

Image	Class
	Yeah, my major is Civil Environmental & Engineering.
	When are you go home?
	See you next time.

C. Data Augmentation

```python
# number of images augment each original image
augment_number = 6


#  pipeline augmentation
transform = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.RandomBrightnessContrast(p=0.2),
    A.Rotate(limit=30, p=0.5),
    A.Blur(p=0.1),
    A.Affine(translate_percent=0.05, scale=(0.95, 1.05), rotate=(-15, 15), p=0.5)
])
```



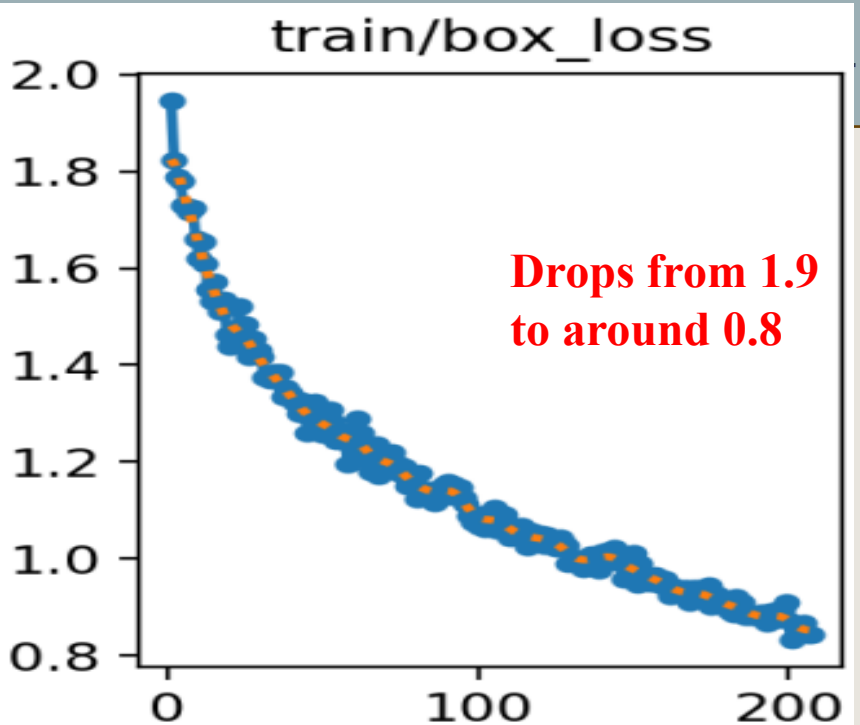
Train	70%
Val	20%
Test	10%

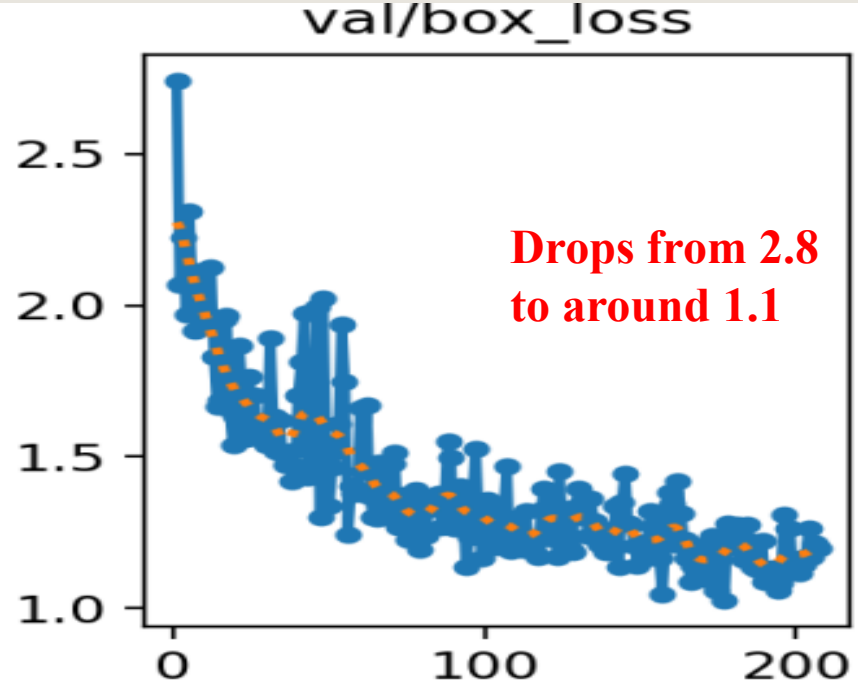
Dimension	Original data	Augmentation data
640 x 480	120	720

III. TRAINING and EVALUATION

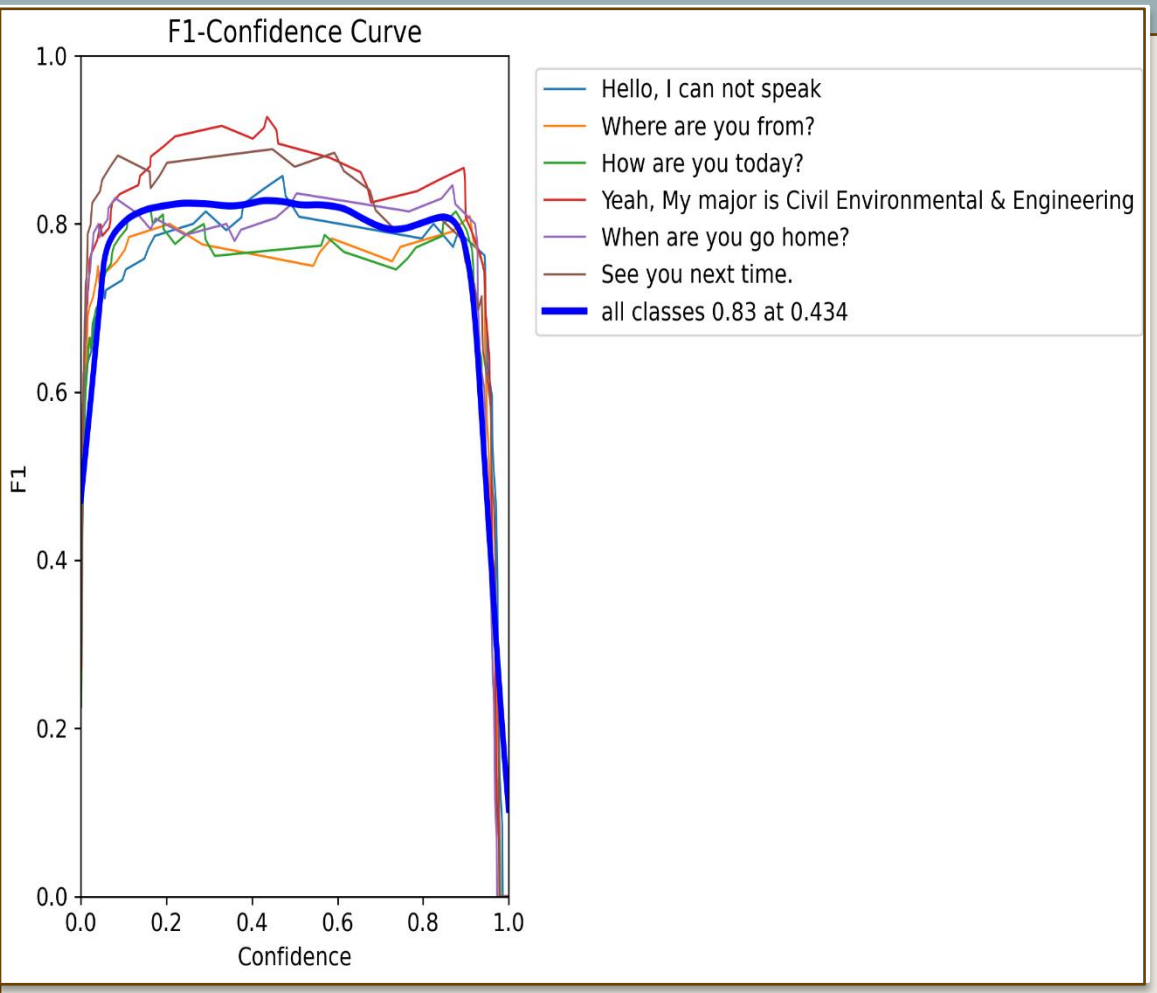
```python
# ================== LOAD BASE MODEL ==================
print("\n" + "=" * 100)
print("LOADING BASE YOLOv11 MODEL...")
print("=" * 100 + "\n")
try:
    model = YOLO(base_model)
    print(f"Loaded model: {base_model}\n")
except Exception as e:
    print(f"Error loading base model '{base_model}': {e}")
    sys.exit(1)

# ================== TRAINING ==================
print("\n" + "=" * 100)
print("START TRAINING YOLOv11 (BASELINE)...")
print("=" * 100 + "\n")

try:
    results = model.train(
        data=yaml_file,
        imgsz=640,
        epochs=200,
        batch=64,
        device=0,
        project=train_output,
        name=run_name,
        exist_ok=True,
        workers=4,
        verbose=True,
    )
```

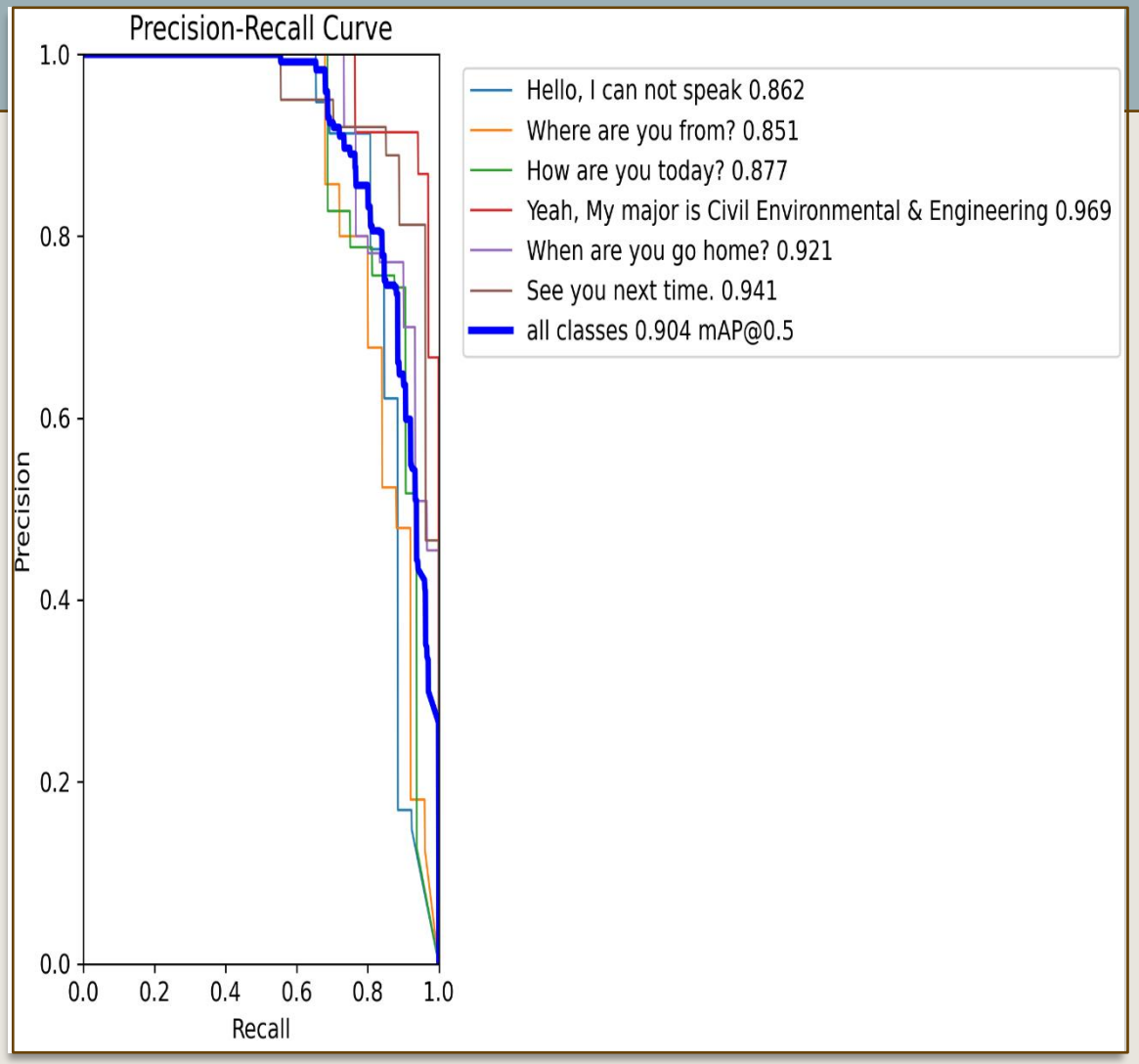
-The model is learning to predict bounding boxes more accurately.





III. TRAINING and EVALUATION





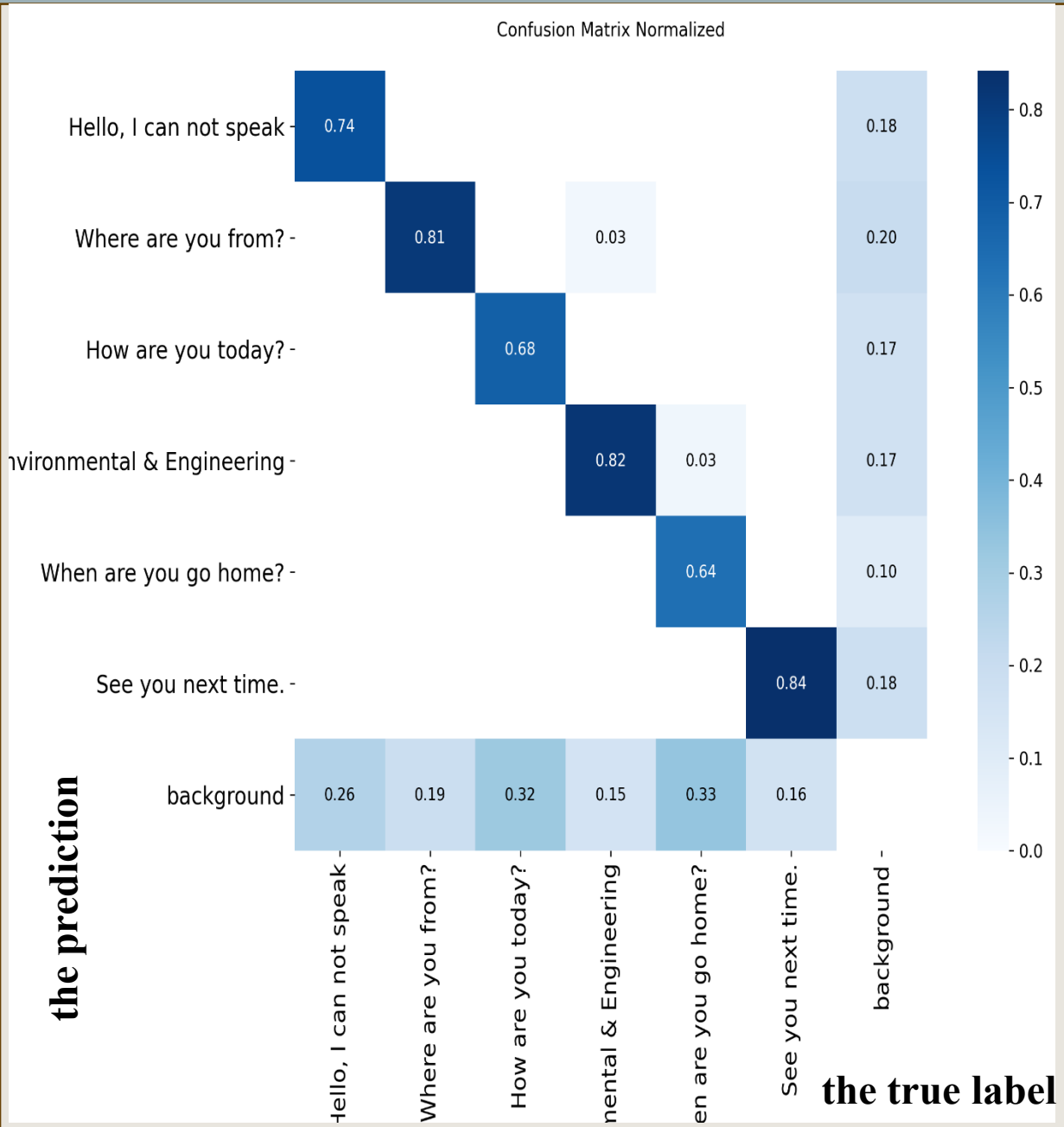
The plot shows that a confidence threshold of about 0.43 is optimal for the system.

- detections with confidence ≥ threshold
- confidence < threshold -> can not detection

All classes have Average Precision between about 0.85 and 0.97, and the overall performance is 0.904 .

=> The model recognizes hand signs with high accuracy and **low error**

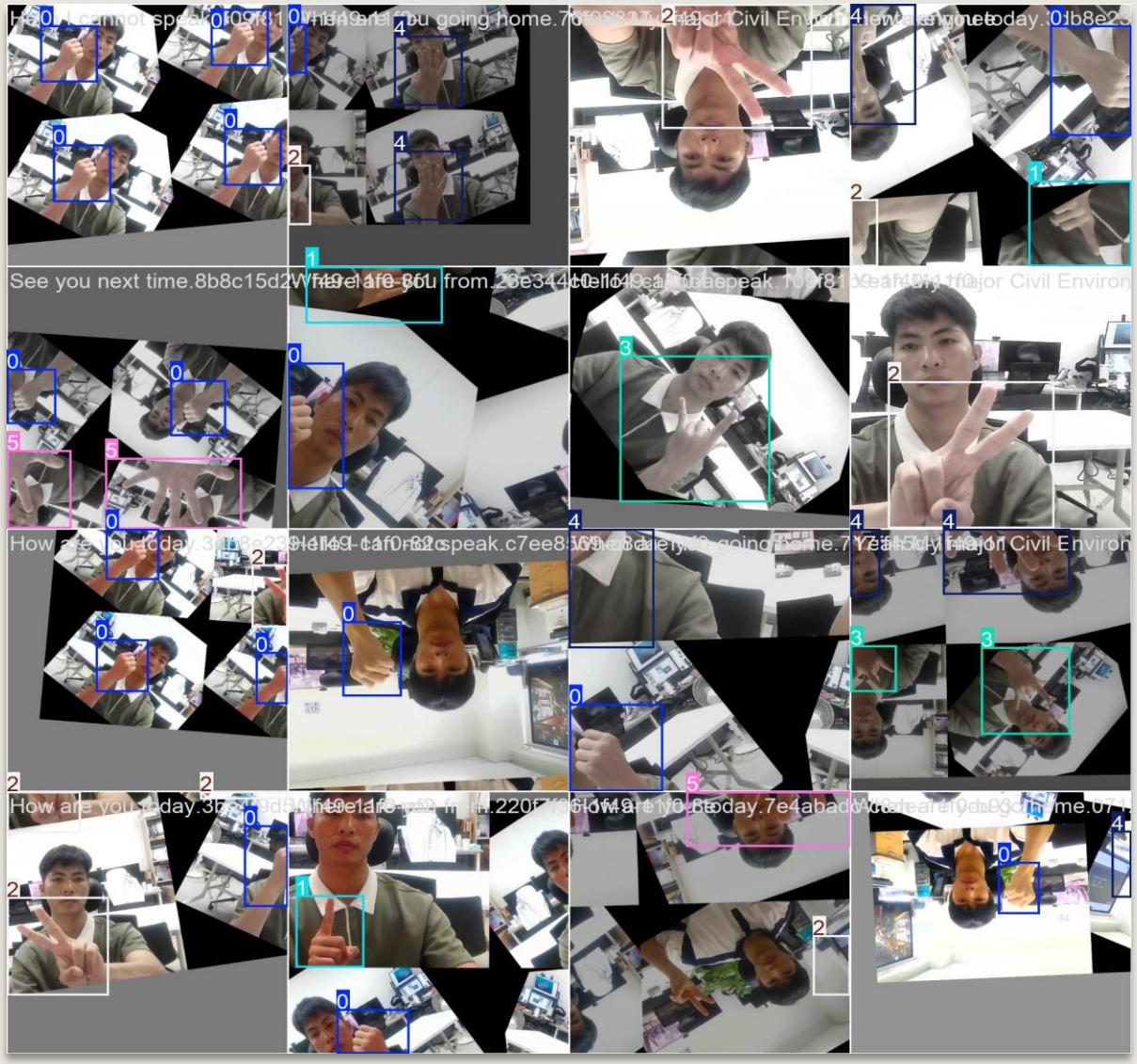
III. TRAINING and EVALUATION



Positive: the high diagonal values about 0.74–0.84 show that most gestures are correctly recognized, with little confusion between different gesture classes.

-**Drawback**: The confusion with the background: roughly 10–20% of GT are predicted as background, and about 15–30% of background frames are misclassified as gestures.

III. TRAINING and EVALUATION



The example detections on test images (correct sentence labels)

III. PREDICTION from WEBCAM

```python
# COCO keypoint index mapping for YOLO pose
LEFT_WRIST_IDX = 9
RIGHT_WRIST_IDX = 10
GESTURE_TEXT = {
    0: 'Hello, I can not speak',
    1: 'Where are you from?',
    2: 'How are you today?',
    3: 'Yeah, my major is Civil Environmental & Engineering',
    4: 'When are you go home?',
    5: 'See you next time.'
}
GESTURE_RESPONSE = {
    0: 'Hi, what can I help you now?',
    1: 'I am from Viet Nam.',
    2: 'I am great, how about you?',
    3: 'Good, do you like your major?',
    4: 'I will go home after 10 minutes.',
    5: 'Yes, have a nice day!'
}
COLORS = {···
PERSON_COLORS = [
    (0, 255, 0),
    (0, 0, 255),
    (255, 0, 0),
    (0, 255, 255),
    (255, 0, 255),
    (255, 255, 0),
    (128, 0, 255),
    (0, 128, 255),
]
```

This block sets up the webcam prediction.

The wrist keypoints for YOLO pose, map each gesture ID (0–5) to a sentence shown on the screen.

III. PREDICTION from WEBCAM


```python
# ---------------- POSE (YOLO11 POSE) ----------------
def extract_pose_people(self, pose_result, conf_thresh=0.25):
    people = []
    if pose_result is None or pose_result.keypoints is None:
        return people
    kpts = pose_result.keypoints
    xy = kpts.xy
    conf = getattr(kpts, 'conf', None)

    if hasattr(xy, 'cpu'):
        xy = xy.cpu().numpy()
    else:
        xy = np.array(xy)

    if conf is not None:
        if hasattr(conf, 'cpu'):
            conf = conf.cpu().numpy()
        else:
            conf = np.array(conf)

    num_people = xy.shape[0]
    for i in range(num_people):
        wrists = []
        lx, ly = xy[i, LEFT_WRIST_IDX]
        lw_conf = conf[i, LEFT_WRIST_IDX] if conf is not None else 1.0
        if lw_conf >= conf_thresh and not (lx == 0 and ly == 0):
            wrists.append((float(lx), float(ly)))

        rx, ry = xy[i, RIGHT_WRIST_IDX]
        rw_conf = conf[i, RIGHT_WRIST_IDX] if conf is not None else 1.0
        if rw_conf >= conf_thresh and not (rx == 0 and ry == 0):
            wrists.append((float(rx), float(ry)))
        if wrists:
            people.append({'id': i, 'wrists': wrists})

    return people
```

```python
def match_hand_to_person(self, det, people):
    if not people:
        return None

    cx = (det['x1'] + det['x2']) / 2.0
    cy = (det['y1'] + det['y2'

    best_id = None
    best_dist = None

    for person in people:
        for (wx, wy) in person['wrists']:
            dx = cx - wx
            dy = cy - wy
            dist = dx * dx + dy * dy
            if best_dist is None or dist < best_dist:
                best_dist = dist
                best_id = person['id']

    return best_id

def get_person_color(self, person_id):
    if person_id is None:
        return COLORS['box']
    return PERSON_COLORS[person_id % len(PERSON_COLORS)]
```

-Get the center of a detected hand box and compares it with all wrist points

=> **Target:** The right-side code links each detected hand to the nearest person and assigns a consistent color for drawing that person in the webcam output

- Takes the **YOLO-pose** result from each webcam frame.
- Reads all keypoints and **converts** them to NumPy arrays.
- For every detected person, it checks the confidence of the left and right wrist.

IV. EXPERIMENTAL RESULT

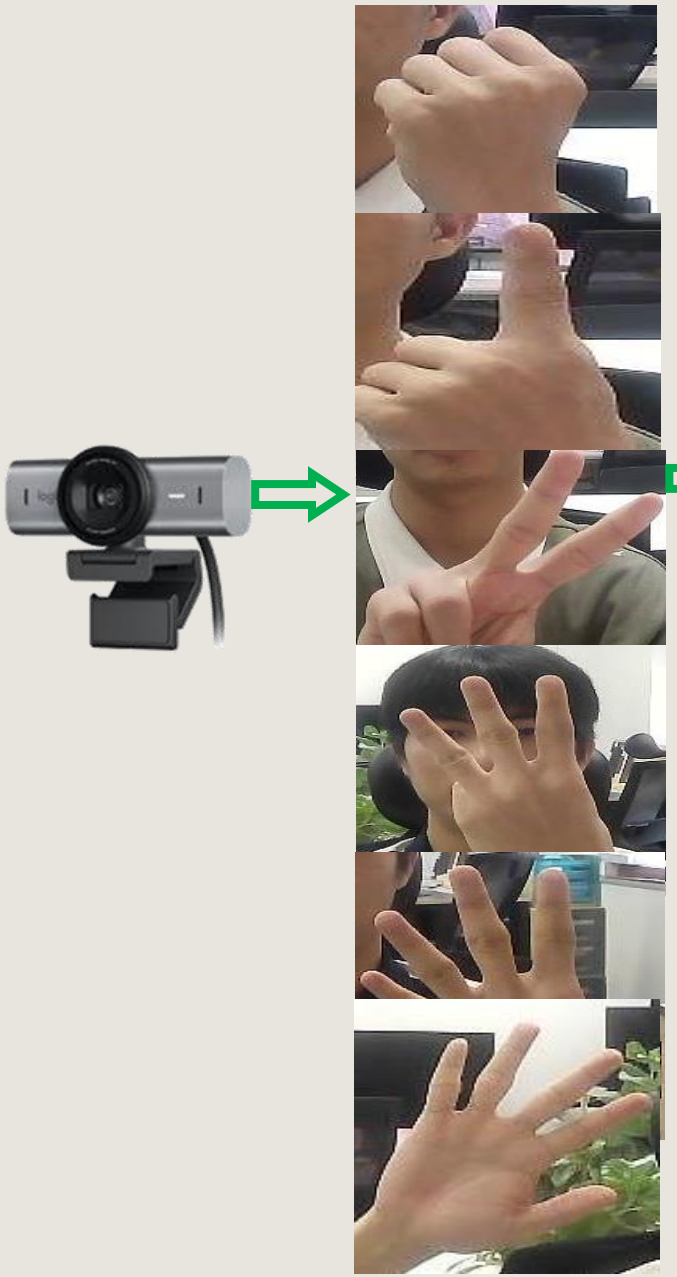
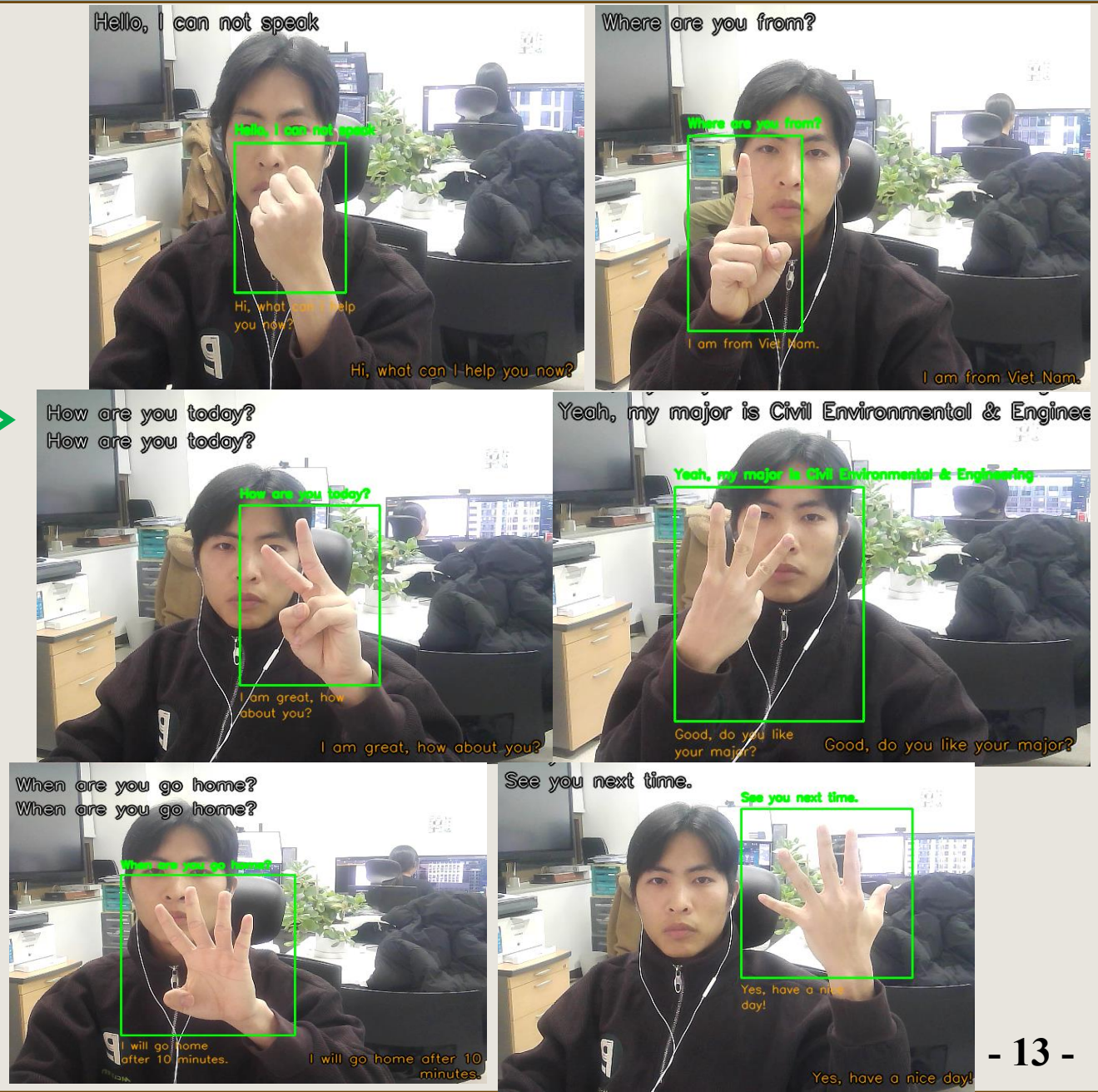
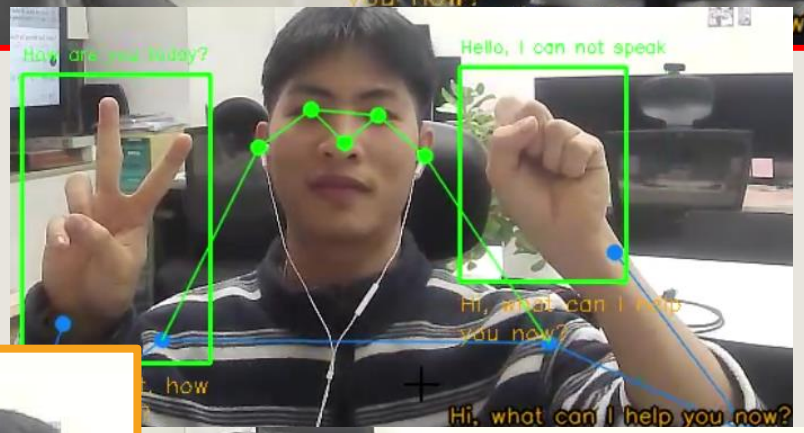


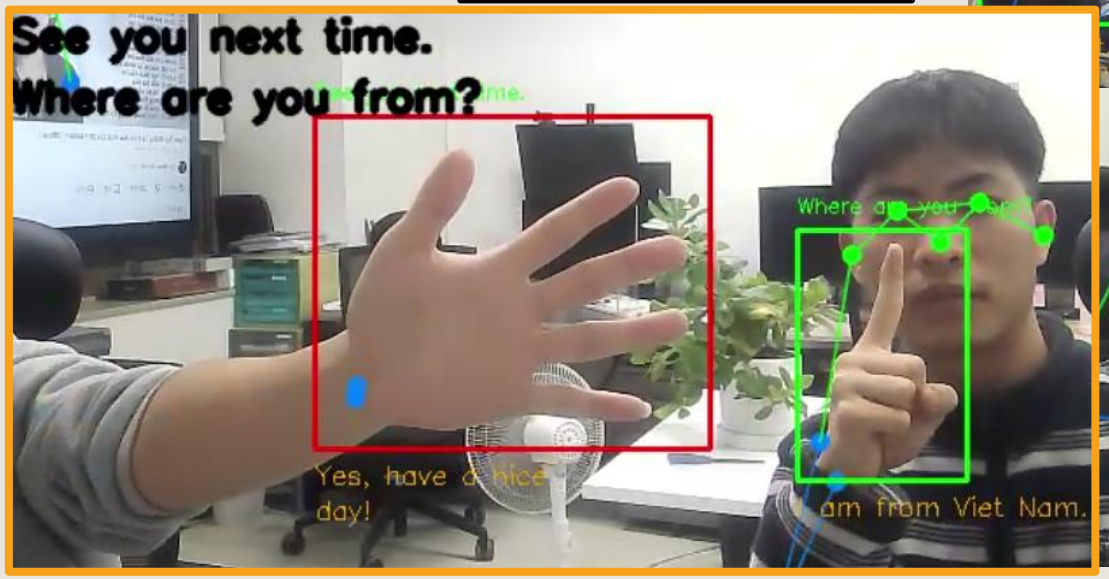
Image processing

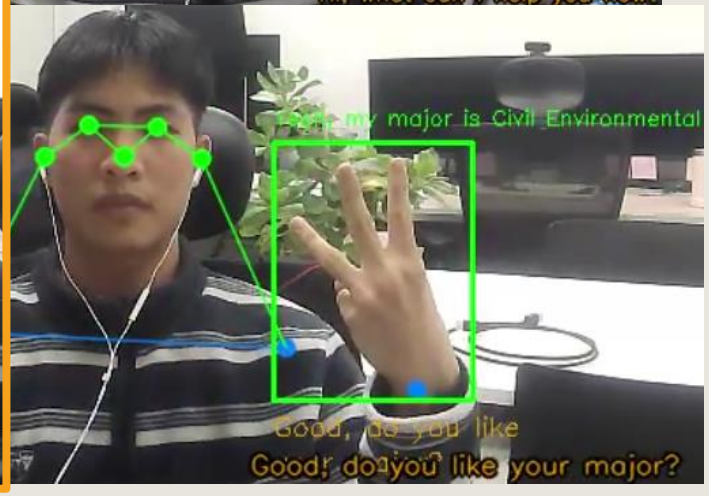


IV. EXPERIMENTAL RESULT

Image processing

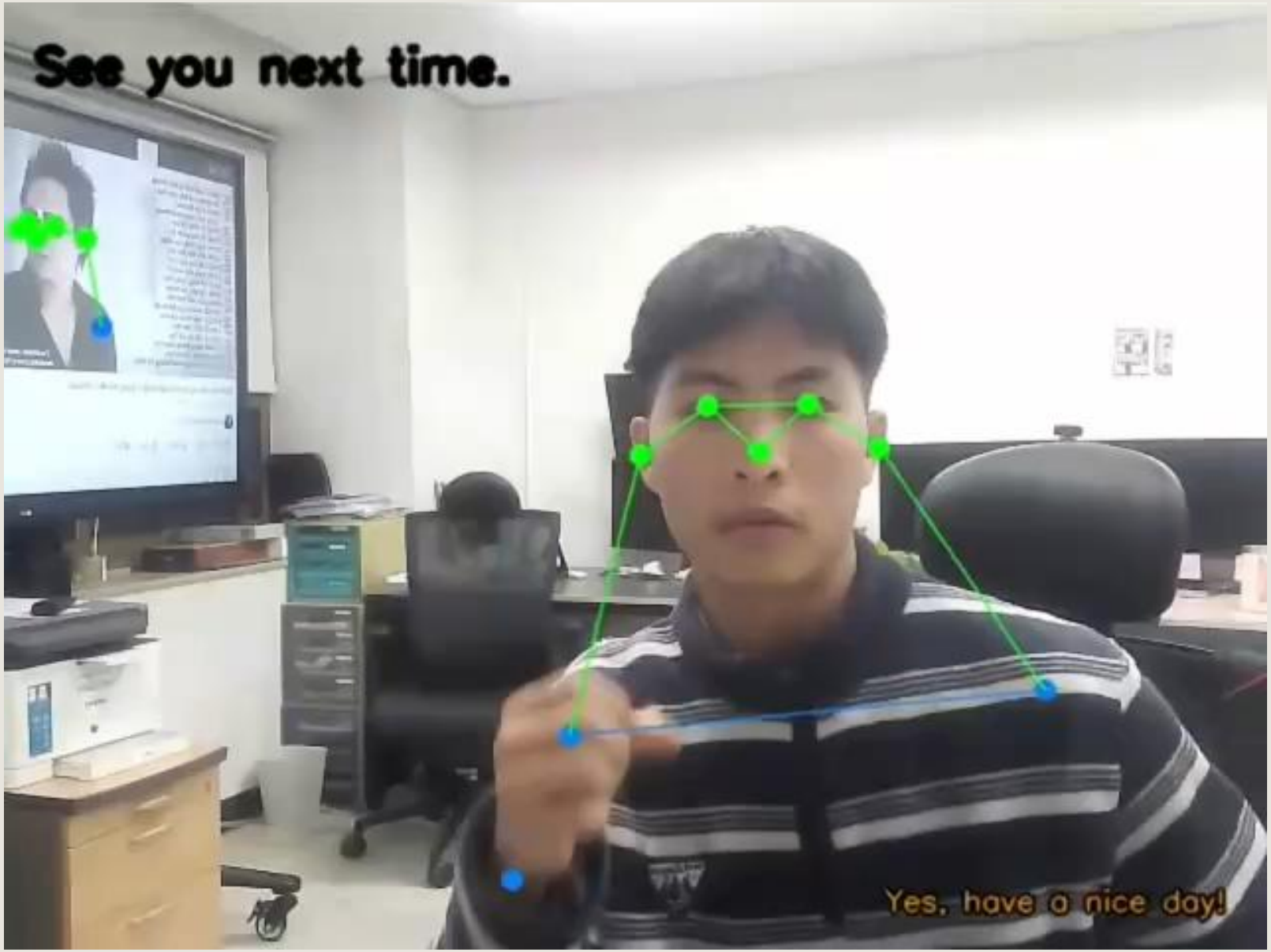






IV. EXPERIMENTAL RESULT

IV. EXPERIMENTAL RESULT VIDEO



IV. CONCLUSION

*** Strengths:**

- Real-time processing: fast inference.
- **Non-invasive interface**: users only need to make hand gestures - no wearables required.
- **Multi-modal output**: delivers both text and speech, adaptable to different contexts.
- **Robust operation:** performs reliably across varying backgrounds, lighting conditions, and viewing angles.
- Support for deaf or mute users: enhances their ability to communicate.

*** Limitations:**

- Lighting dependency: recognition quality degrades under very bright or very dark conditions.
- Limited generalization: not yet tested extensively across diverse hand sizes, skin tones, and clothing.
- Gesture similarity confusion: very similar gestures can sometimes be misclassified.

Thank You!